

Parallel Workflows in LangGraph | Agentic AI using LangGraph

1. Introduction

- A **parallel workflow** runs multiple independent tasks at the same time.
- Unlike a sequential workflow:

Sequential: $A \rightarrow B \rightarrow C$

Parallel: $A \rightarrow (B + C + D) \rightarrow E$

- Parallel execution can reduce total execution time when tasks do not depend on each other.

2. Installation

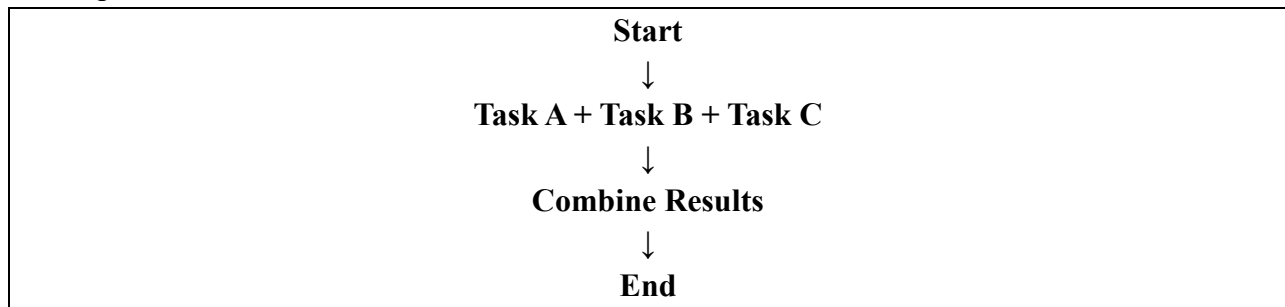
- Set up LangGraph and the required Python dependencies.
- Create a virtual environment to keep project dependencies isolated.
- Install the required LLM/provider packages if the workflow uses an LLM.

3. First Workflow

A basic parallel workflow contains:

- **State** — Shared workflow data.
- **Nodes** — Individual tasks.
- **Edges** — Define how execution moves between tasks.

Example:

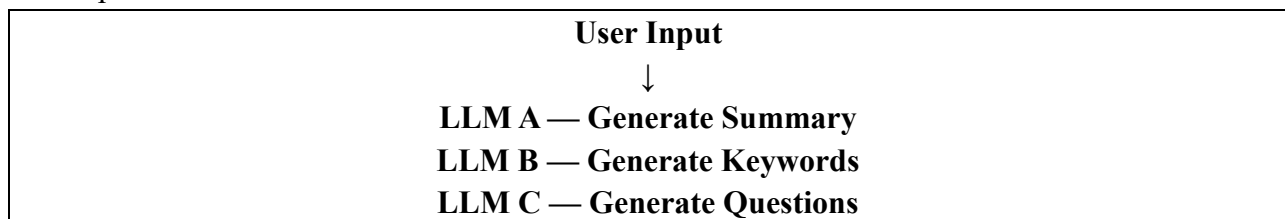


The independent nodes can execute in parallel.

4. LLM Workflow

Multiple LLM tasks can also run in parallel.

Example:



↓
Combine Results

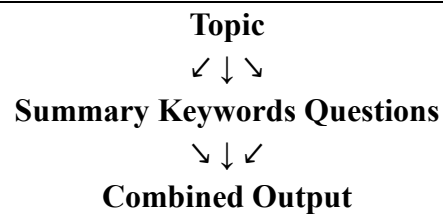
Because these tasks are independent, they don't need to wait for one another.

Benefit: Multiple pieces of information can be generated efficiently from the same input.

5. Prompt Chaining

- Prompt chaining normally follows a **sequential** pattern where one LLM's output becomes the next LLM's input.
- In a parallel workflow, independent prompts can instead receive the **same input simultaneously**.

Example:



The important distinction is:

- **Sequential chaining:** Output of A → Input of B
- **Parallel execution:** Same input → Multiple independent tasks

Key takeaway: Use parallel workflows when tasks are **independent**, then combine their results when they are all complete.